

Mentor : Mr Arnol Namegni

- **Projet 6** ●

**Prédiction de la consommation énergétique
des bâtiments non résidentiels de Seattle**

Candidat : Mr Jean-Francois Chaussoy

Alternant Data Engineering



Ville neutre en carbone d'ici 2050



Situation :

Élément	Détail
Client	Ville de Seattle
Objectif	Neutralité carbone en 2050
Problème	Les relevés énergétiques sont coûteux et rares
Enjeu	Prédire la consommation sans faire les mesures

Ce que nous allons prédire :

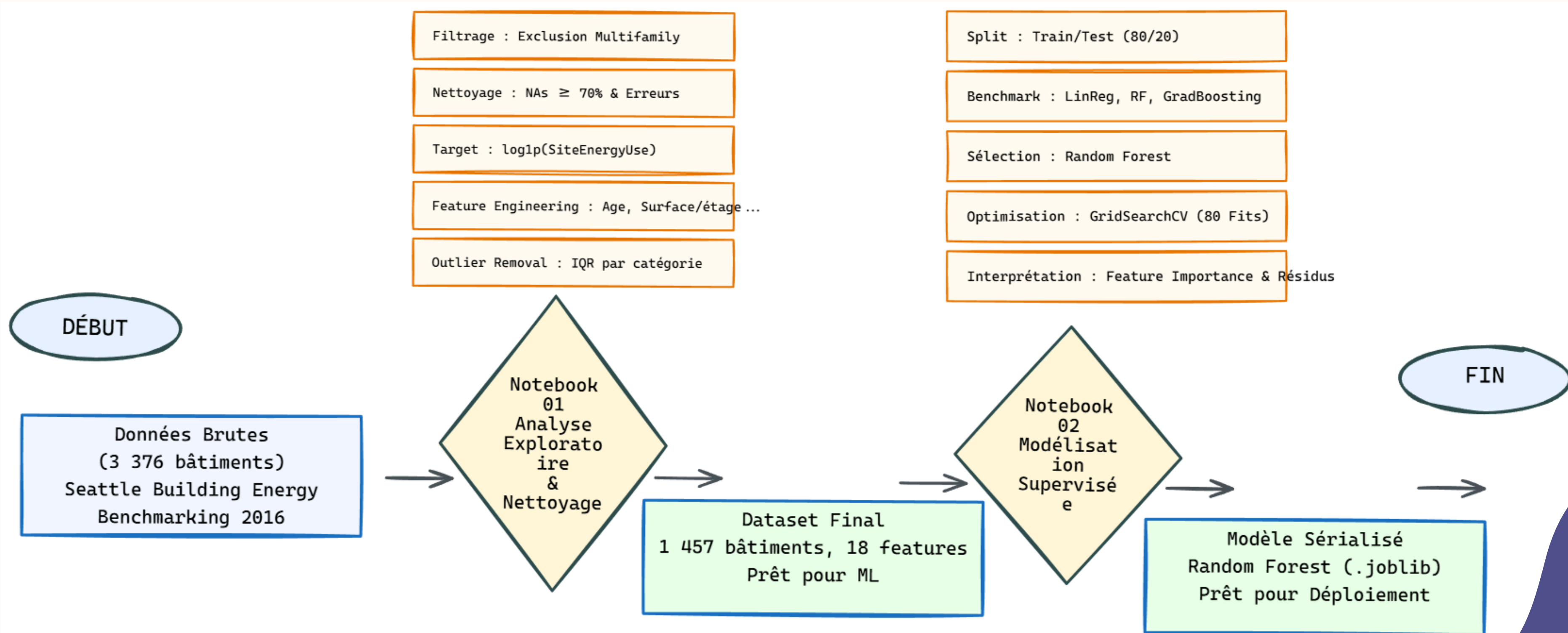
SiteEnergyUse(kBtu) — Consommation totale d'énergie du bâtiment, en **kBtu/an**

Données structurelles utilisées :

- Taille du bâtiment (surface, étages)
- Type d'usage (bureau, école, hôtel...)
- Année de construction

 Les données de consommation par source (gaz, électricité) sont exclues — les utiliser serait du data leakage.

Déroulé du projet



Données : Seattle Building Energy Benchmarking 2016



● Vue d'ensemble :

3 376 lignes x 46 colonnes au départ

Étape	Lignes avant	Lignes après	Raison
Suppression colonnes (≥70% NA)	46 colonnes	41 colonnes	Comments, Outlier, YearsENERGYSTARCertified...
Filtrage bâtiments résidentiels	3 376	1 668	Multifamily LR/MR/HR hors périmètre
Suppression types résiduels mal classés	1 668	1 627	Residence Hall, Senior Care

Source : City of Seattle Open Data Portal
(2016)

● Répartition finale par catégorie

Catégorie	Bâtiments
Office	507
Industrial & Storage	290
Other	253
Education	163
Retail & Food	143
Mixed Use	114
Hospitality	76
Public / Institutional	71
Healthcare	10

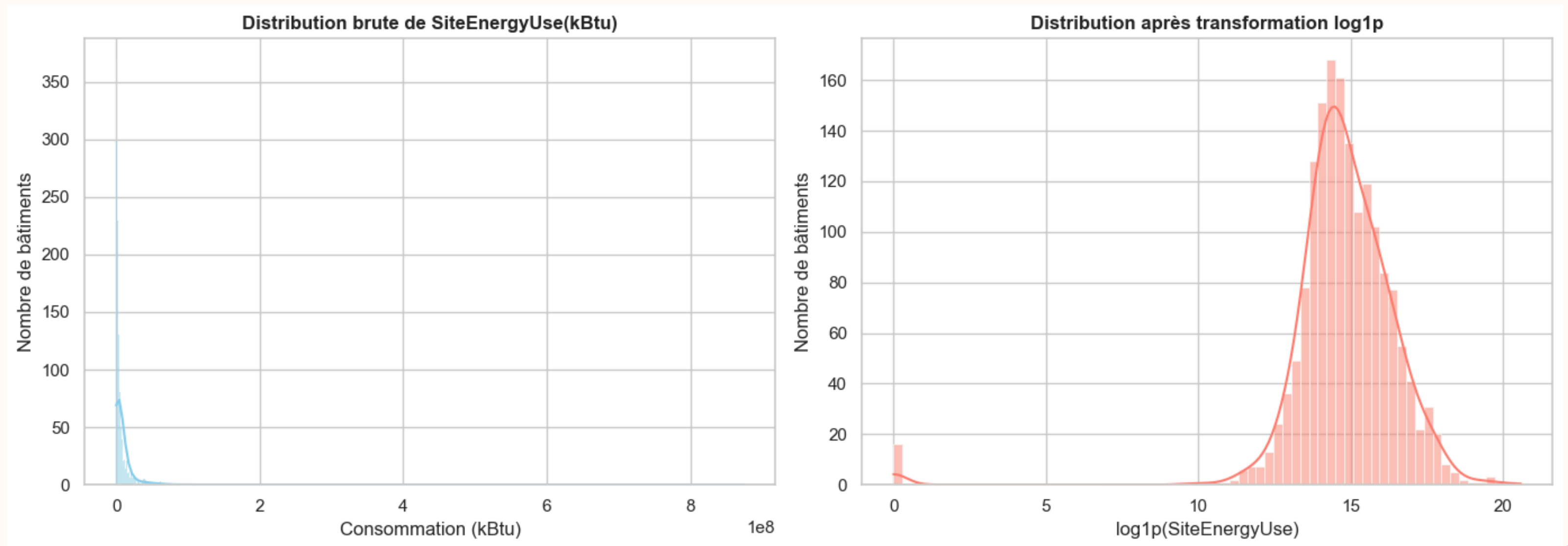


Ce que les données nous apprennent :

Insight 1 : La cible est très asymétrique → transformation $\log_{10}$

Distribution brute : forte asymétrie à droite (quelques bâtiments = consommation extrême)

Distribution $\log_{10}$: proche d'une loi normale → favorable aux modèles de régression



on modélise **$\log_{10}(\text{SiteEnergyUse})$** et on décode avec **expm1** en fin de pipeline



Décision :

On modélise $\log_{10}(\text{SiteEnergyUse})$ et on décode avec `expm1` en fin de pipeline

Insight 2 : La surface est le principal déterminant

Feature	Corrélation avec la cible (brute)
PropertyGFATotal	0.79
PropertyGFABuilding(s)	0.76
LargestPropertyUseTypeGFA	0.74
NumberofFloors	0.42

Insight 3 : La consommation varie fortement selon le type

- Healthcare (hôpitaux) : consommation la plus élevée (fonctionnement 24h/24)
- Industrial & Storage : très variable (entrepôts frigorifiques vs. self-storage)
- Office (bureaux) : profil le mieux représenté, consommation modérée



Pourquoi faire du feature engineering ?

Les données brutes ne racontent pas tout.

On crée de nouvelles variables pour donner plus de contexte métier au modèle.

4 nouvelles features créées

Feature	Formule	Intuition métier
BuildingAge	2016 - YearBuilt	Un bâtiment ancien est moins bien isolé
SurfaceParEtage	PropertyGFATotal / NumberofFloors	Densité d'occupation par étage
RatioLargestUse	LargestPropertyUseTypeGFA / PropertyGFATotal	Bâtiment mono-usage vs. multi-usage
PropertyCategory	Regroupement de 19 types → 8 catégories	Réduire la cardinalité pour l'encodage

Encodage de la variable catégorielle

PropertyCategory (8 catégories) → One-Hot Encoding

Catégorie de référence supprimée : "Education" (drop_first=True)

Résultat : 8 colonnes binaires (0 / 1)



Nettoyage avant modélisation

Pourquoi par catégorie ?

Un hôpital qui consomme 50 millions de kBtu est normal pour un hôpital, mais serait un outlier extrême pour un entrepôt.

Résultats du nettoyage IQR

Catégorie	Avant	IQR Supprimés	%
Education	163	22	13.5%
Office	507	50	9.9%
Industrial & Storage	290	26	9.0%
Mixed Use	114	16	14.0%
Hospitality	76	10	13.2%
Healthcare	10	0	0.0%
Toutes catégories	1 627	161	9.9%

Dataset final pour modélisation

- 1 457 bâtiments × 18 features
- Train (80%) : 1 165 bâtiments
- Test (20%) : 292 bâtiments
- random_state = 42 (reproductibilité)



3 modèles testés — Cross-validation 5 folds

Modèle	MAE	CVRMSE	CVR ²	CVR ² Train
Régression Linéaire	0.93	1.79	0.13	0.23
Random Forest	0.80	1.78	0.14	0.87
Gradient Boosting	0.80	1.85	0.07	0.77

(Métriques sur les valeurs log1p de la cible — validation croisée 5 folds sur le jeu d'entraînement)



Observation importante : sur-apprentissage du Random Forest

Modèle	R ² Train	R ² Test (CV)	Écart
Random Forest	0.88	0.14	Fort
Gradient Boosting	0.77	0.07	Fort
Régression Linéaire	0.23	0.13	Stable

 Le Random Forest mémorise les données d'entraînement mais généralise peu. Ce phénomène s'appelle l'overfitting. Il est lié à la taille réduite du dataset (~1 165 bâtiments en train) et à la forte variance naturelle de la consommation énergétique.



Malgré tout : Random Forest retenu pour la MAE la plus faible

La MAE = 0.80 (log1p) du RF est la meilleure parmi les 3 modèles
il commet les erreurs les moins importantes en valeur absolue.

Optimisation du modèle (GridSearch)



Affinage du Random Forest

Hyperparamètres explorés via GridSearchCV

```
325 param_grid = {
326     "model__n_estimators": [300, 600], # Nombre d'arbres
327     "model__max_depth": [None, 20], # Profondeur max
328     "model__min_samples_split": [2, 5], # Nb min de samples pour diviser un
    nœud
329     "model__min_samples_leaf": [1, 2], # Nb min de samples par feuille
330     "model__max_features": ["sqrt"], # Sous-ensemble aléatoire des variables
331 }
332 # Total : 16 combinaisons x 5 folds = 80 entraînements
```

Meilleurs paramètres trouvés

- n_estimators = 300
- max_depth = None (profondeur illimitée)
- min_samples_split = 2
- min_samples_leaf = 2
- max_features = 'sqrt' (réduit l'overfit en décorrélant les arbres)

Résultats avant / après tuning (jeu de test)

Métrique	Avant tuning	Après tuning
R ² (log1p)	0.14	0.22
RMSE (log1p)	1.53	1.46
MAE (log1p)	0.74	0.71

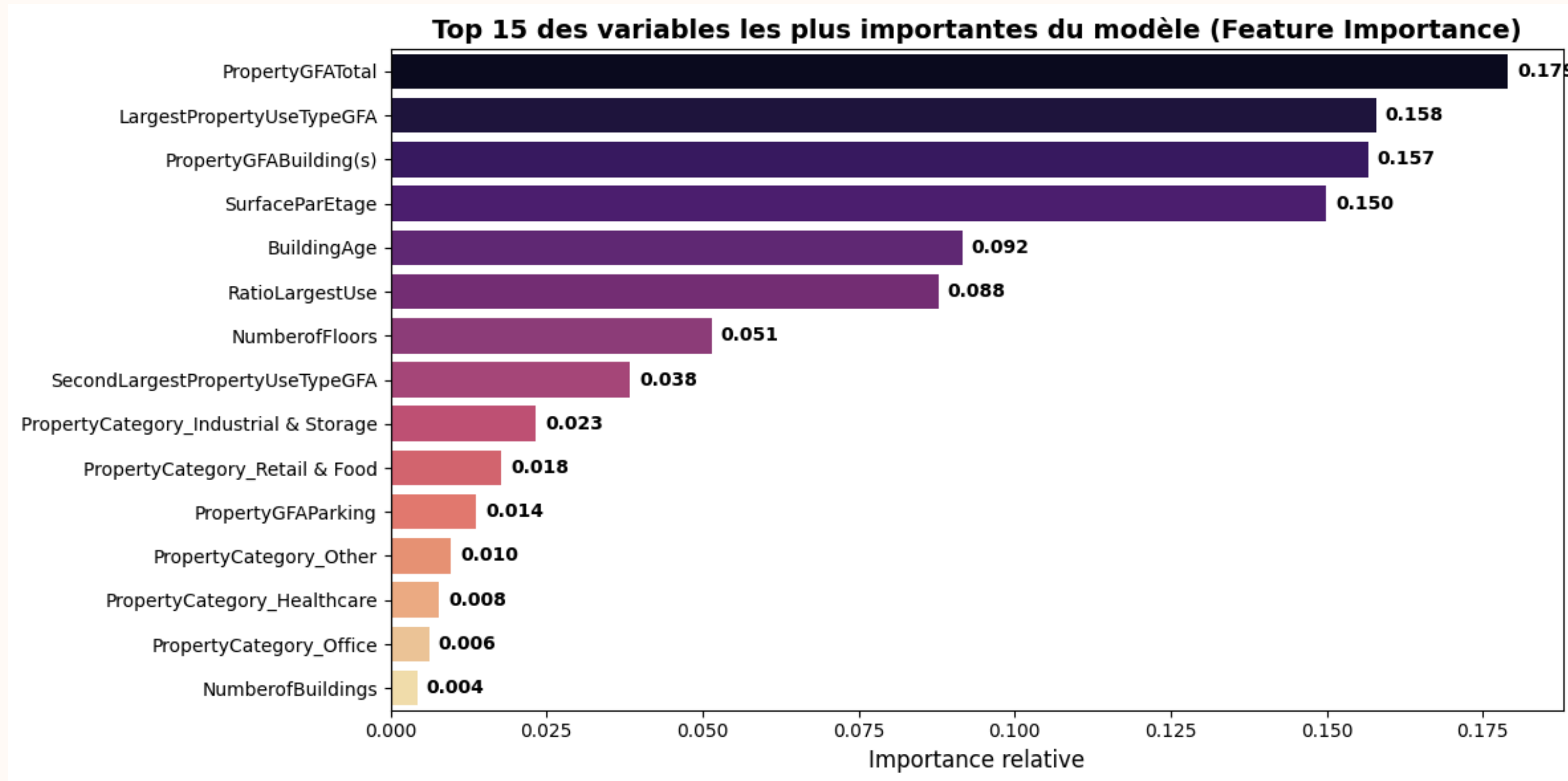
R² passe de 0.14 à 0.22 et RMSE baisse de 1.53 à 1.46. Néanmoins, l'écart avec les performances d'entraînement persiste en raison d'une limite structurelle des données.

Feature Importance



Qu'est-ce qui influence le modèle ?

Top 15 des variables les plus importantes avec étiquettes de coefficients textuels

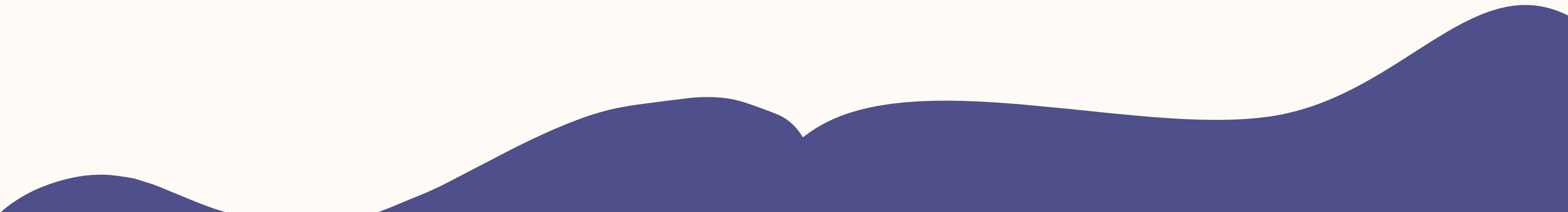




Interprétation métier

Feature	Importance	Signification
Surface totale	27.6%	La taille est le 1er déterminant
Surface par étage (<i>feature créée</i>)	18.1%	La densité compte autant que la taille brute
Surface 1er usage	17.5%	Usage principal = 2e signal le plus fort
Ratio usage (<i>feature créée</i>)	9.6%	L' usage mixte influence la conso
Âge bâtiment (<i>feature créée</i>)	7.0%	L' isolation a un impact réel

3 des 5 features les plus importantes sont des features créées

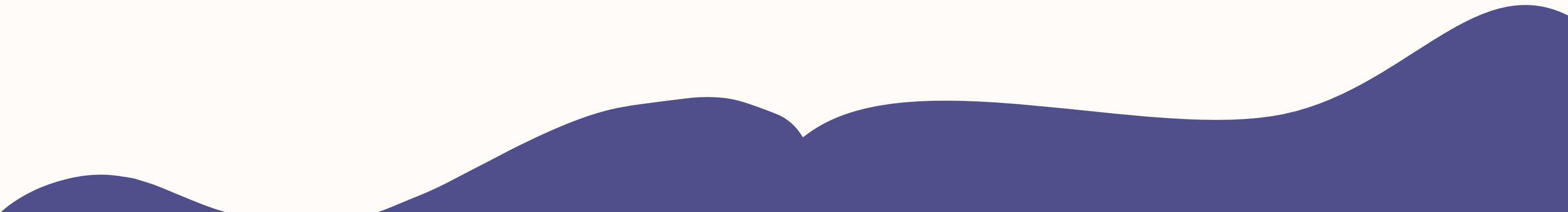




Interprétation métier

Feature	Importance	Signification
PropertyGFATotal	27.6%	La taille est le 1er déterminant
SurfaceParEtage	18.1%	La densité compte autant que la taille brute
LargestPropertyUseTypeGFA	17.5%	Usage principal = 2e signal le plus fort
RatioLargestUse	9.6%	L'usage mixte influence la conso
PropertyGFABuilding(s)	7.1%	
BuildingAge	7.0%	L'isolation a un impact réel
SecondLargestPropertyUseTypeGFA	4.7%	
PropertyCategory_Industrial&Storage	2.8%	
PropertyCategory_Retail & Food	1.5%	
PropertyCategory_Healthcare	1.4%	

3 des 5 features les plus importantes sont des features créées — le feature engineering a apporté de la valeur.





Récapitulatif

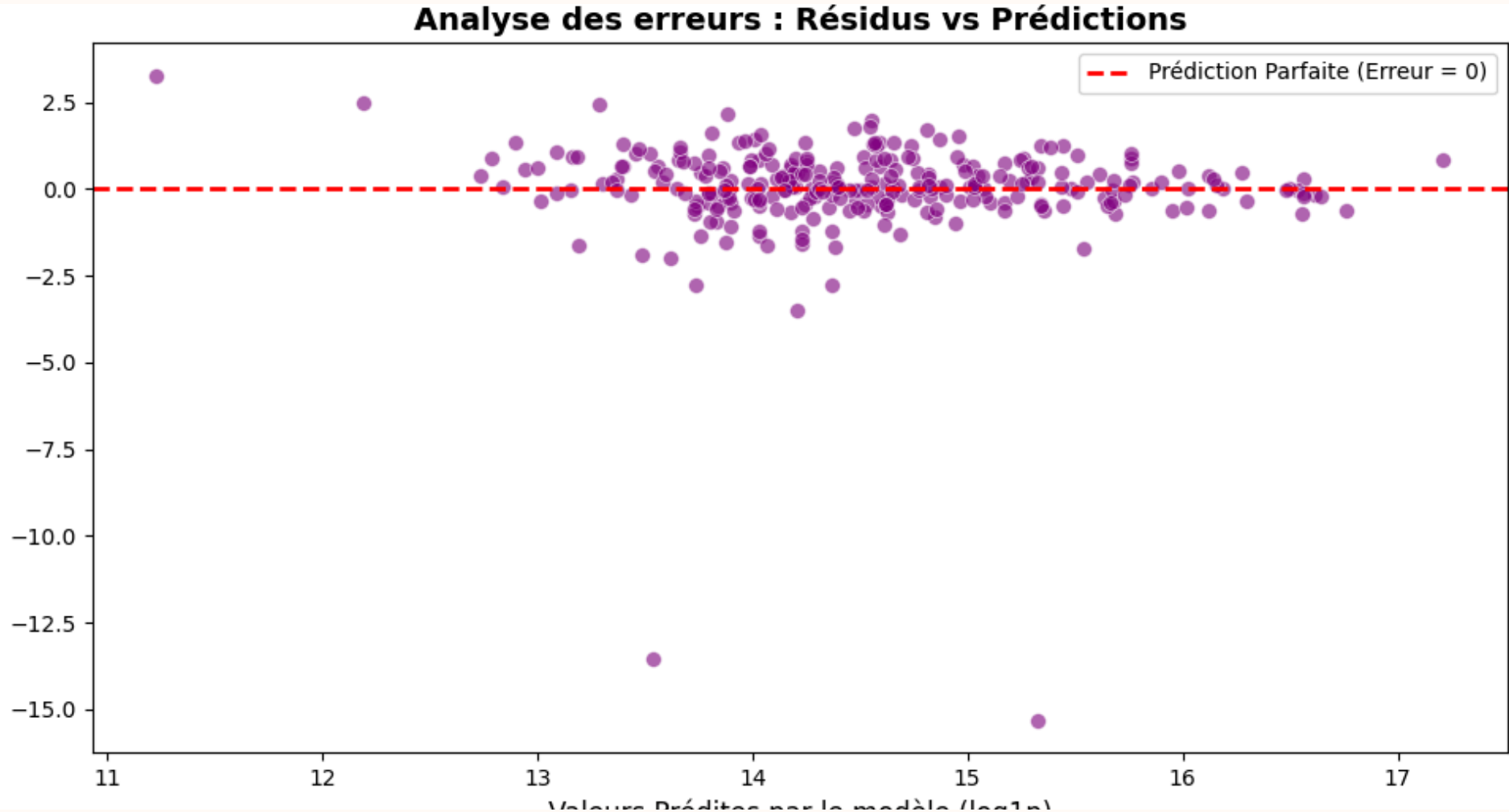
Forces	Limites identifiées
3 features créées figurent dans le Top 5	Fort overfitting (R^2 train 0.88 vs test 0.14)
Data leakage évité rigoureusement	Dataset limité (~1 457 bâtiments)
Validation croisée rigoureuse (5 folds)	Forte variance naturelle de la cible
API déployée et fonctionnelle	Le modèle sous-prédit les gros bâtiments



Forces et limites

Point	Action recommandée
Overfitting	Réduire la complexité du modèle (plus de régularisation) ou augmenter les données
Dataset petit	Data augmentation synthétique ou chercher plus de données publiques
Sous-prédiction gros bâtiments	Ajouter une feature "taille catégorie" ou entraîner un modèle spécifique pour les gros bâtiments
Features créées utiles	Continuer à engineer des features basées sur la géométrie et l'usage

Synthèse et bilan



Démo live : L'API de prédiction



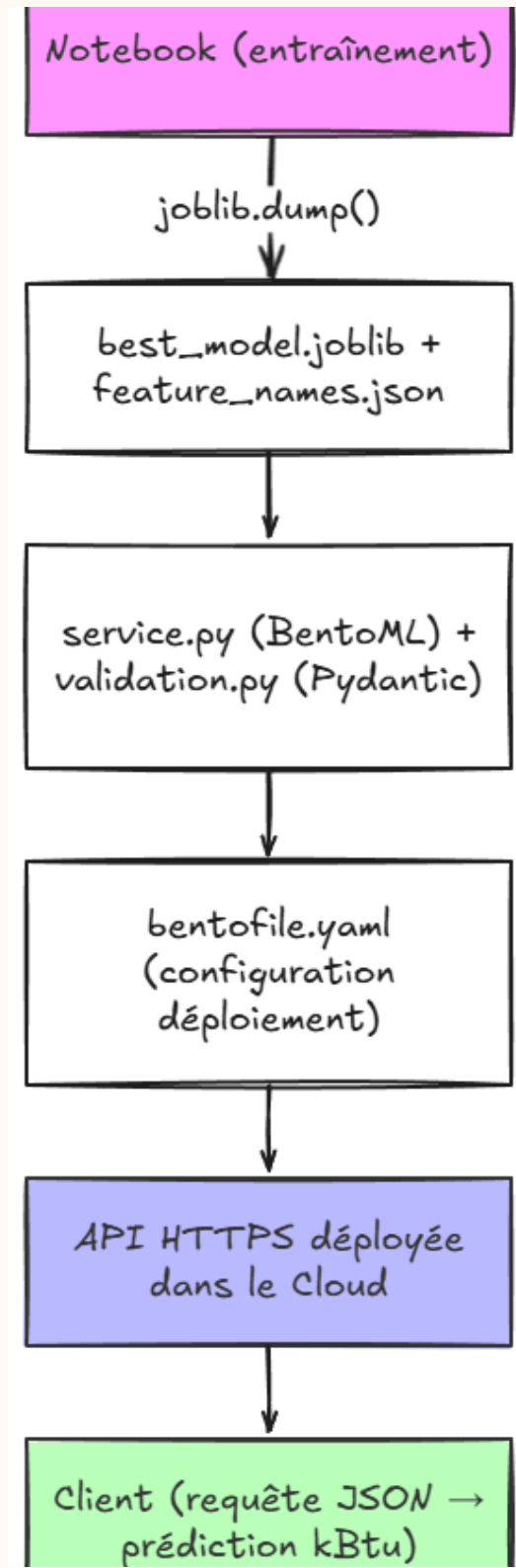
Architecture du déploiement local

Architecture du déploiement AWS

<http://51.21.3.82:3000>

Endpoints disponibles

Endpoint	Méthode	Description
/predict	POST	Prédit la consommation (kBtu)
/healthcheck	GET	Vérifie que le service tourne





Exemple de requête /predict

```
535 POST /predict
536 {
537   "primary_property_type": "Large Office",
538   "year_built": 1995,
539   "number_of_buildings": 1,
540   "number_of_floors": 10,
541   "property_gfa_total": 150000,
542   "property_gfa_parking": 20000,
543   "property_gfa_building": 130000,
544   "largest_property_use_type_gfa": 125000,
545   "second_largest_property_use_type_gfa": 0
546 }
```

Réponse attendue

```
535 {
536   "predicted_energy_kbtu": 4823156.42,
537   "predicted_log1p_energy": 15.3891,
538   "unit": "kBtu",
539   "input_summary": {
540     "property_type": "Large Office",
541     "year_built": 1995,
542     "total_area_sqft": 150000
543   }
544 }
```


Validation des données d'entrée



Architecture de validation

- COUCHE 1 — Types Python (automatique)
year_built doit être un entier (int) → Erreur automatique si "mil neuf cent" est envoyé.
- COUCHE 2 — Contraintes de valeurs
year_built : compris entre 1800 et 2016.
number_of_floors : compris entre 1 et 200.
property_gfa_total : > 0 et ≤ 5 000 000.
- COUCHE 3 — Cohérence Métier
La surface du bâtiment (property_gfa_building) additionnée à la surface du parking (property_gfa_parking) doit être égale à la surface totale (property_gfa_total).



Démonstration — Données invalides rejetées

Donnee	Erreur_retournee
year_built = 2030	Valeur doit être ≤ 2016
property_gfa_building > property_gfa_total	GFA building ne peut pas dépasser GFA total
primary_property_type = "Villa"	Type non reconnu. Valeurs acceptées : [...]

Clôture

Ce que nous avons accompli



Livrable	Statut
Analyse exploratoire + Feature Engineering	Notebook complet
Comparaison 3 modèles supervisés (CV 5 folds)	RandomForest retenu
Optimisation GridSearch (18 combinaisons)	Meilleurs params identifiés
Modèle sauvegardé (best_model.joblib)	Sauvegardé
API BentoML déployée avec /predict et /healthcheck	Déployée
Validation Pydantic en 3 couches	Opérationnelle
Tests API (test_api.py)	Écrits



Merci pour votre attention !

